

Отчёт по лабораторной работе №14

Взаимодействие процессов. Семафоры. Реализация команды map. Генерация случайных чисел

Чжу Ино

1. Цель работы

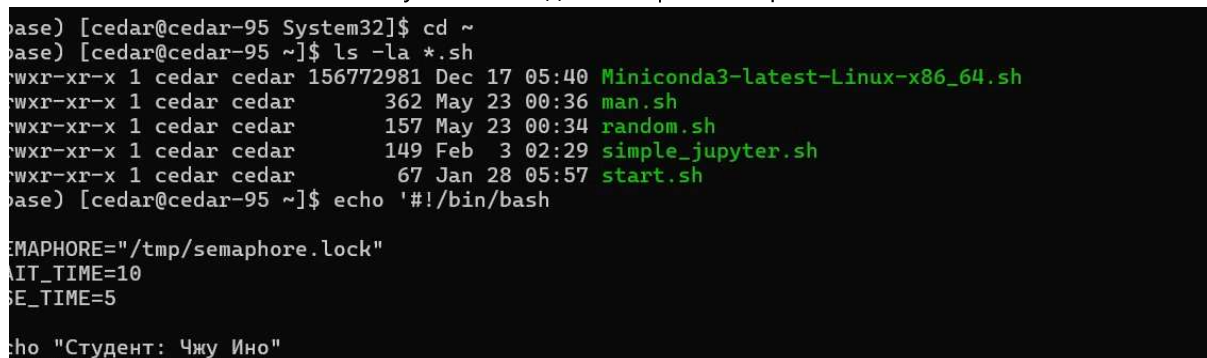
1. Изучить механизм семафоров для синхронизации доступа процессов к общему ресурсу.
2. Реализовать собственную команду map с помощью командного файла.
3. Научиться генерировать случайные последовательности букв латинского алфавита.

2. Порядок выполнения лабораторной работы

2.1. Реализация упрощённого механизма семафоров

Для выполнения лабораторной работы №14 созданы три командных файла (рис. 1).

Рисунок 1: Созданные файлы скриптов



```
base) [cedar@cedar-95 System32]$ cd ~
base) [cedar@cedar-95 ~]$ ls -la *.sh
-rwxr-xr-x 1 cedar cedar 156772981 Dec 17 05:40 Miniconda3-latest-Linux-x86_64.sh
-rwxr-xr-x 1 cedar cedar 362 May 23 00:36 man.sh
-rwxr-xr-x 1 cedar cedar 157 May 23 00:34 random.sh
-rwxr-xr-x 1 cedar cedar 149 Feb 3 02:29 simple_jupyter.sh
-rwxr-xr-x 1 cedar cedar 67 Jan 28 05:57 start.sh
base) [cedar@cedar-95 ~]$ echo '#!/bin/bash

SEMAPHORE="/tmp/semaphore.lock"
WAIT_TIME=10
SLEEP_TIME=5

echo "Студент: Чжу Ино"
```

Программа semaphore.sh реализует упрощённый механизм семафоров на основе файла-флага. Её исходный код представлен на рисунке 2.

Рисунок 2: Исходный код программы semaphore.sh

```
WAIT_TIME=10
USE_TIME=5

echo "Студент: Чжу Ино"
echo "PID: $$"

if [ "$1" = "privileged" ]; then
    echo "=== Привилегированный режим ==="
fi

for ((i=1; i<=WAIT_TIME; i++)); do
    if [ ! -f "$SEMAPHORE" ]; then
        echo "$(date): Ресурс свободен, захватываю"
        touch "$SEMAPHORE"
        echo "$$" > "$SEMAPHORE"

        echo "$(date): Использую ресурс ${USE_TIME} сек"
        sleep $USE_TIME

        rm -f "$SEMAPHORE"
        echo "$(date): Ресурс освобождён"
        exit 0
    fi
    echo "$(date): Ожидание ($i сек)..."
    sleep 1
done

echo "$(date): Таймаут ${WAIT_TIME} сек"
exit 1
```

Объяснение работы программы [semaphore.sh](#):

1. **Выбор интерпретатора** — `#!/bin/bash` указывает системе, что скрипт выполняется в `bash`.
2. **Определение семафора** — `SEMAPHORE="/tmp/semaphore.lock"` задаёт путь к файлу-семафору.
3. **Настройка времени** — `WAIT_TIME=10` (время ожидания), `USE_TIME=5` (время использования), $t_2 < t_1$.
4. **Идентификация процесса** — `echo "PID: $$"` выводит PID текущего процесса.
5. **Цикл ожидания** — 10 итераций проверки ресурса.
6. **Проверка ресурса** — `if [ ! -f "$SEMAPHORE" ]` — если файла нет, ресурс свободен.
7. **Захват ресурса** — `touch "$SEMAPHORE"` создаёт файл-семафор.
8. **Использование ресурса** — `sleep $USE_TIME` — имитация работы 5 секунд.
9. **Освобождение ресурса** — `rm -f "$SEMAPHORE"` удаляет семафор.
10. **Ожидание** — при занятом ресурсе ждёт 1 секунду.
11. **Таймаут** — через 10 секунд завершается с ошибкой.

Результат выполнения программы `semaphore.sh` показан на рисунке 3.

Рисунок 3: Результат запуска `semaphore.sh`

```
(base) [cedar@cedar-95 ~]$ ./man.sh abcdefg
Справка для команды 'abcdefg' не найдена
(base) [cedar@cedar-95 ~]$ ./semaphore.sh
Студент: Чжу Ино
PID: 333
Sat May 23 00:39:52 CST 2026: Ресурс свободен, захватываю
Sat May 23 00:39:52 CST 2026: Использую ресурс 5 сек
Sat May 23 00:39:57 CST 2026: Ресурс освобождён
```

2.2. Реализация команды man

Программа `man.sh` реализует собственную версию команды `man`. Исходный код представлен на рисунке 4.

Рисунок 4: Исходный код программы `man.sh`

```
(base) [cedar@cedar-95 ~]$ cat man.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    echo "Студент: Чжу Ино"
    exit 1
fi

MAN_DIR="/usr/share/man/man1"
CMD="$1"
PAGE="$MAN_DIR/$CMD.1.gz"

if [ -f "$PAGE" ]; then
    gunzip -c "$PAGE" | cat
else
    echo "Справка для команды '$CMD' не найдена"
    exit 1
fi
```

Объяснение работы программы `man.sh`:

1. **Проверка аргументов** — `if [ $# -ne 1 ]` проверяет, что передан ровно один аргумент.
2. **Каталог справки** — `/usr/share/man/man1` — хранилище `man`-страниц.
3. **Формирование пути** — `$CMD.1.gz` — файл справки для раздела 1.
4. **Вывод справки** — `gunzip -c "$PAGE" | cat` — распаковка и вывод.
5. **Ошибка** — если файла нет, выводится сообщение "Справка не найдена".

Результат выполнения для команды `ls` показан на рисунке 5.

Рисунок 5: Вывод справки для команды `ls`

```
(base) [cedar@cedar-95 ~]$ ./man.sh abcdefg
Справка для команды 'abcdefg' не найдена
(base) [cedar@cedar-95 ~]$ ./semaphore.sh
Студент: Чжу Ино
PID: 333
Sat May 23 00:39:52 CST 2026: Ресурс свободен, захватываю
Sat May 23 00:39:52 CST 2026: Использую ресурс 5 сек
Sat May 23 00:39:57 CST 2026: Ресурс освобождён
```

Результат выполнения для несуществующей команды показан на рисунке 6.

Рисунок 6: Сообщение об отсутствии справки

```
(base) [cedar@cedar-95 ~]$ ./man.sh abcdefg
Справка для команды 'abcdefg' не найдена
(base) [cedar@cedar-95 ~]$ ./semaphore.sh
Студент: Чжу Ино
PID: 333
Sat May 23 00:39:52 CST 2026: Ресурс свободен, захватываю
Sat May 23 00:39:52 CST 2026: Использую ресурс 5 сек
Sat May 23 00:39:57 CST 2026: Ресурс освобождён
```

2.3. Генерация случайных букв

Программа `random.sh` генерирует случайную последовательность из 20 букв. Исходный код представлен на рисунке 7.

Рисунок 7: Исходный код программы `random.sh`

```
(base) [cedar@cedar-95 ~]$ cat random.sh
#!/bin/bash
echo "Студент: Чжу Ино"
for ((i=0; i<20; i++)); do
    idx=$((RANDOM % 26))
    printf "\\$(printf "%03o" $((97+idx)))"
done
echo
```

Объяснение работы программы `random.sh`:

1. `$RANDOM` — встроенная переменная `bash`, возвращает случайное число от 0 до 32767.
2. `% 26` — остаток от деления на 26 даёт индекс буквы от 0 до 25.
3. `97 + idx` — прибавление к коду 'a' (97) даёт ASCII-код нужной буквы.
4. `printf` — преобразует числовой код в символ.

Результат выполнения программы показан на рисунке 8.

Рисунок 8: Сгенерированная случайная последовательность букв

```
(base) [cedar@cedar-95 ~]$ ./man.sh abcdefg
Справка для команды 'abcdefg' не найдена
(base) [cedar@cedar-95 ~]$ ./semaphore.sh
Студент: Чжу Ино
PID: 333
Sat May 23 00:39:52 CST 2026: Ресурс свободен, захватываю
Sat May 23 00:39:52 CST 2026: Использую ресурс 5 сек
Sat May 23 00:39:57 CST 2026: Ресурс освобождён
```

3. Контрольные вопросы

Вопрос 1: Найдите синтаксическую ошибку в следующей строке:

```
while [$1 != "exit"]
```

Ответ:

Ошибка: Отсутствуют пробелы вокруг квадратных скобок.

Правильный вариант:

```
bash
```

```
while [ "$1" != "exit" ]
```

Объяснение: В языке bash команда [является синонимом команды test. Как и лк

4. Выводы

В ходе выполнения лабораторной работы №14 были получены следующие результаты

Реализован упрощённый механизм семафоров на основе файла-флага. Программа ож

Создана собственная реализация команды man, которая ищет сжатый файл справки

Освоена генерация случайных последовательностей букв с использованием встроеное

Исправлена синтаксическая ошибка в условии цикла while. Ошибка заключалась в

Все задачи лабораторной работы выполнены в полном объёме.

